# IDX Financial Statement Extraction & Wren AI Platform

A Complete, Plain-English Engineering Deep-Dive

Architecture · Schema · Code Walkthrough · Wren AI & Chat Agent · Debugging Playbook · Interview Prep

Project: **statements-extraction**

Prepared for: engineers presenting or onboarding onto this project

Reading level: written for a junior engineer — no prior context assumed

Source of truth: this document was written by reading the actual source code, not just the design docs, so it reflects what the system really does today.

# Table of Contents

**1** What Is This Project, and Why Does It Exist?

**2** The Big Picture: Architecture in One Story

**3** Project Structure Tour (folder by folder)

**4** The Data Journey: From PDF to Database, Step by Step

**5** The Database Schema, Explained Table by Table

**6** The Command-Line Interface (CLI) Reference

**7** Going Distributed: Redis, Celery, and the Staging Lake

**8** The FastAPI Gateway

**9** Wren AI: Talking to the Data in Plain English

**10** Inside chat_agent.py: How the Chat Agent Really Works

**11** How to Debug This Project: A Practical Playbook

**12** Interview Preparation Guide

**13** Glossary of Terms

**14** Appendix: Full File Map

Tip: open the PDF outline / bookmarks panel in your PDF viewer (usually a sidebar icon) to jump straight to any section or sub-section — every heading in this document is a clickable bookmark.

# 1. What Is This Project, and Why Does It Exist?

## 1.1 The problem in one sentence

Indonesian public companies (issuers on the Indonesia Stock Exchange, "IDX" / "BEI") publish two big PDF documents every reporting period: an audited **Financial Statement** ("Laporan Keuangan" / LK, think: balance sheet, income statement, cash flow) and a **Annual Report** ("Laporan Tahunan" / AR, a 400+ page narrative document with management commentary, KPIs, governance profiles, and the audited statements reprinted at the back). Nobody wants to manually copy numbers out of hundreds of PDFs for dozens of companies every quarter — so this project reads those PDFs with an LLM, turns the numbers into clean structured data, checks that the accounting actually adds up, stores it in a proper database, and finally lets you *ask questions about it in plain English* through a chatbot.

## 1.2 Why not just use a normal scraper or regex?

Because every company writes things slightly differently. One company calls revenue "Pendapatan Usaha", another calls it "Penjualan dan Pendapatan Usaha", a bank calls it "Pendapatan Bunga" (interest income), a mining contractor calls it something else entirely. A pure regex/rules approach turns into an endless game of whack-a-mole as you add more companies and industries. This project instead uses an LLM to *read and understand* each document like a junior analyst would, but it never lets the LLM do arithmetic or decide what the final "truth" is — that job stays with deterministic Python code. This split is the single most important design idea in the whole codebase, and it shows up everywhere.

> **The one rule to remember:** the LLM finds and labels things (it is *probabilistic*), but plain Python code parses numbers, does the math, and checks that the books balance (it is *deterministic*). If you remember only one sentence from this document, remember this one — it explains almost every design decision that follows.

## 1.3 What "done" looks like today

The pipeline has been run end-to-end against **14+ real Indonesian companies** across 11 different industries (banking, nickel mining, coal, palm oil, retail, aviation, IT services, renewable energy, gold mining, agrochemicals, marine logistics), producing **3,325+ normalized numeric facts** in a database, with **100% of "hard" accounting rules passing** (e.g. the balance sheet always balances to the last cent/rupiah). On top of that database sits a natural-language chat layer (built on "Wren AI") so a non-technical analyst can type a question like *"Berapa laba bersih ESSA tahun 2024 dan 2025?"* ("What was ESSA's net profit in 2024 and 2025?") and get a grounded, cited answer instead of a hallucinated one.

## 1.4 Who would use this, and how

| Role | What they do with it |
|---|---|

| | |
|---|---|
| Data/ML engineer | Runs the extraction pipeline on new PDFs, tunes prompts and validation rules, maintains the taxonomy. |
| Equity research analyst | Queries the Gold layer (SQL or chatbot) to compare companies, check ratios, and trace numbers back to a PDF page. |
| Software/platform engineer | Runs and scales the Celery/Redis/FastAPI services, keeps the DuckDB warehouse healthy, wires up Wren AI. |
| Reviewer / auditor | Uses the "provenance" views to check that every number can be traced back to an exact PDF page. |

# 2. The Big Picture: Architecture in One Story

## 2.1 The "Medallion" mental model

The whole system follows a very common data-engineering pattern called the **Medallion Architecture**: data gets progressively cleaner as it moves through named layers, like refining ore into metal.

| Layer | Nickname | What lives here |
|---|---|---|
| `idxfin_ref` | Reference | The "dictionary": 148 standard accounting concepts and label-to-concept mappings. Rarely changes. |
| `idxfin_bronze` | Raw | Exactly what was ingested: the PDF's identity (hash, ticker, page count) and raw page text. Nothing is cleaned yet. |
| `idxfin_silver` | Cleaned | Every extracted number as one row ("fact"), fully normalized, scaled, and tagged with exactly which PDF page it came from. |
| `idxfin_market` | External | Stock prices, commodity prices (gold, oil, ammonia), and FX rates pulled from Yahoo Finance — kept separate from company filings because it comes from a different, external source. |
| `idxfin_gold` | Analytics-ready | Pre-built views anyone can query directly: "financial_summary", "market_valuation", etc. This is what the chatbot and analysts actually use. |

Think of it like cooking: Bronze is the raw grocery haul, Silver is chopped and prepped ingredients, Gold is the finished, plated dish ready to serve.

## 2.2 The seven phases, end to end

Reading the design docs (`docs/DESIGN_LOG.md`) and the actual code together, the system has seven phases:

1. **Ingest & sniff metadata** — open the PDF, hash it, and guess the ticker/currency/scale/language from the cover pages (`src/parser/pdf_loader.py`).

2. **Sectionize** — slice the PDF into meaningful chunks: "this is the balance sheet, pages 4-17", "this is the notes on borrowings, pages 69-88" (`src/parser/sectionizer.py`).
3. **Extract via LLM** — for each section, call an LLM with a strict schema (Pydantic) so it can only answer in a shape the code understands (`src/extraction/*.py`, `src/llm/*.py`).
4. **Normalize** — parse the raw number strings into real Decimal values, apply the document's scale (millions/thousands/etc.), and map each row label to a standard concept ID (`src/normalization/*.py`, `src/mapping/concept_mapper.py`).
5. **Validate** — run ~33 accounting equations (does Assets = Liabilities + Equity?) and flag or auto-repair anything that doesn't add up (`src/validation/*.py`).
6. **Store** — write to Parquet first (safe, parallel-friendly), then load into DuckDB through a single serialized writer (`src/storage/*.py`).
7. **Serve** — expose Gold views through SQL, a FastAPI HTTP gateway, and a natural-language chat agent powered by Wren AI (`src/api/*`, `wren/*`).

## 2.3 Two ways to run it

The project supports both a simple, synchronous mode (great for a laptop, one PDF at a time) and a fully distributed mode (great for processing hundreds of filings overnight):

| Mode | How it runs | When to use it |
|---|---|---|
| Synchronous CLI | `python3 -m src.cli extract file.pdf` runs the whole pipeline in-process and writes straight to DuckDB. | Local development, debugging a single filing, demos. |
| Distributed (Celery) | FastAPI enqueues a job to Redis; 6 worker processes pull PDFs off the queue, extract in isolation, and write to a Parquet "staging" folder; a separate single-writer worker loads Parquet into DuckDB. | Bulk/batch processing many filings, or running as a hosted service. |

## 2.4 Why the two modes need different plumbing: the DuckDB concurrency problem

DuckDB is an **embedded** database — it's a library that runs inside your process, like SQLite, not a server you connect to over the network. That makes it fast and simple, but it means only **one process** can open it for writing at a time. If you tried to run 6 Celery workers that each open `data/idxfin.duckdb` for read-write directly, you'd get a crash like:

```
duckdb.IOException: Could not set lock on file "data/idxfin.duckdb": Resource temporarily unavailable
```

The fix used here is a classic "two-phase commit" pattern:

- **Phase 1 — stateless extraction:** each of the 6 extraction workers processes one PDF fully in isolation (calls the LLM, normalizes, validates) and writes its results as **Parquet files** into `data/staging/pending/<run_id>/`. No worker ever touches the DuckDB file directly. Parquet is just a column-oriented file format — think of it as a very efficient CSV that many tools can read fast.
- **Phase 2 — single-writer ingestion:** a dedicated Celery queue called `duckdb_writer` runs with **concurrency = 1** (only one worker, ever) and is additionally protected by a file lock (`FileLock("data/idxfin.duckdb.lock")`). It reads the staged Parquet files with DuckDB's native `read_parquet(...)` (this takes well under 50ms) and inserts them.

> **Why this matters for an interview or a design review:** this is a textbook example of separating "compute" (many parallel workers doing expensive LLM calls) from "commit" (one single, fast, safe writer). The same pattern shows up in log-structured merge trees, in how Spark writes to Delta Lake, and in how most "single-writer" embedded databases are used safely from multiple processes.

## 2.5 Why Parquet staging is also a safety net

Because every extraction run's output is saved as Parquet before it ever touches DuckDB, you get a free disaster-recovery story: if the DuckDB file ever gets corrupted or you need to rebuild it from scratch, you don't need to re-run any LLM calls (which cost money and time) — you just replay the Parquet files with `python3 -m src.cli reindex-staging`.

# 3. Project Structure Tour (folder by folder)

Here is the project laid out, with a one-line "what is this for" next to each important folder or file. Use this as your map before diving into any single file.

## 3.1 Top level

| Path | What it is |
|---|---|
| `schema_design/` | The "contract" layer: what every extractor is allowed to output (Pydantic models), the taxonomy of 148 accounting concepts, and the raw SQL DDL for the database. |
| `src/` | All the pipeline implementation code: PDF parsing, LLM calls, normalization, validation, storage, the CLI, the FastAPI app, Celery tasks. |
| `wren/` | The Wren AI "GenBI" (Generative Business Intelligence) layer: the chat agent script, the semantic model definitions, and Docker compose files for the full Wren microservice stack. |
| `tests/` | 88 automated tests (see Section 11) covering number parsing, validation rules, all-issuer integration checks, Wren semantic contracts, and Celery/staging behaviour. |
| `scripts/` | Small operational helper scripts: batch-dispatch many tickers to Celery, sync market data, organize input folders. |
| `data/` | Where the actual DuckDB file (`idxfin.duckdb`) and the Parquet staging lake (`staging/pending`, `staging/completed`, `staging/failed`) live. This is generated output, not source code. |
| `examples/`, `complete-ticker/` | Sample input PDFs (financial statements + annual reports) for a set of real IDX tickers, used for testing and demos. |
| `docs/` | Design documents: `DESIGN_LOG.md` (deep architecture notes), the LLM-first and Wren AI implementation plans, and diagram sources. |
| `docker-compose.yml`, `Dockerfile` | Runs the core pipeline (Redis, FastAPI, 6 extraction workers, 1 DuckDB-writer worker) as containers. |
| `analysis.ipynb` | A Jupyter notebook for ad-hoc exploration of the DuckDB warehouse. |

## 3.2 Inside `src/` (the pipeline)

| Module | Responsibility |
|---|---|
| `config.py` | Reads `.env`, decides which LLM provider to use (OpenAI / Gemini / OpenRouter), and centralizes all paths and Celery/Redis settings. |
| `pipeline.py` | The orchestrator — `ExtractionPipeline.run()` calls every stage in order. This is the single best file to read first. |
| `cli.py` | The command-line entry point: `inspect`, `extract`, `gold`, `market`, `api`, `worker`, `status`, `reindex-staging`. |
| `parser/pdf_loader.py` | Opens the PDF with PyMuPDF, extracts text per page, and sniffs ticker/currency/scale/locale/doc type from the cover pages. |
| `parser/sectionizer.py` | Splits the document into "sections" (statements, notes, AR chapters) and tags each with a section type used for routing. |
| `llm/client.py` | A provider-agnostic HTTP client (OpenAI / Gemini / OpenRouter) with retries, fallback models, and Pydantic-schema-enforced structured output. |
| `llm/prompts.py` | All prompt templates, including the "candidate concept injection" technique explained in Section 4.4. |
| `extraction/router.py` | Looks at each section's type and dispatches it to the right specialized extractor. |
| `extraction/statements.py` | Extracts face statements (Balance Sheet, Income Statement, Cash Flow, Equity changes). |
| `extraction/kpis.py`, `governance.py`, `assertions.py`, `audit.py` | Extract operational KPIs, board/officer profiles, forward guidance, and the independent auditor's report respectively. |
| `normalization/number_parser.py` | Deterministic string → Decimal parsing: handles locale (comma vs. period), parentheses-as-negative, dashes-as-nil, and the "grounded()" verbatim check. |
| `normalization/normalizer.py` | Turns one LLM extraction result into many `SilverFact` rows, computing a stable hash ID for each fact. |
| `mapping/concept_mapper.py` | Loads the 148-concept taxonomy, holds a ~250-entry dictionary of known label → concept mappings, and falls back to an LLM (with persistence) for anything new. |

| | |
|---|---|
| `validation/engine.py` | Runs the accounting equations and the page-grounding check; can call an LLM to explain and verify unusual reconciling items. |
| `validation/repair.py` | Turns failed hard rules into "review tasks" for a human to look at. |
| `storage/database.py` | Owns the DuckDB connection, schema creation, and the Parquet-to-DuckDB ingestion transaction. |
| `storage/gold_views.py` | Defines every Gold-layer SQL view (financial_summary, market_valuation, etc.) as plain `CREATE OR REPLACE VIEW` statements. |
| `storage/staging.py` | Writes/reads the Parquet staging lake used for the two-phase commit. |
| `market_data/yfinance_loader.py` | Pulls 4 years of daily stock/commodity/FX prices from Yahoo Finance. |
| `celery_app.py`, `tasks.py` | Celery app configuration (two queues, backpressure settings) and the two task functions (extract, ingest). |
| `api/app.py`, `routes.py`, `schemas.py` | The FastAPI application, its HTTP routes, and the request/response Pydantic models. |

## 3.3 Inside `wren/` (the conversational layer)

| File | Purpose |
|---|---|
| `chat_agent.py` | A standalone Python CLI chatbot: LangGraph ReAct agent + Wren's semantic query engine, no Docker required. See Section 10. |
| `config.yaml` | LiteLLM configuration for the full Docker-based Wren AI service: which model to use, embeddings, and the Qdrant vector store. |
| `docker-compose.wren.yml` | Spins up the 5-service Wren AI stack (UI, Engine, AI Service, Qdrant, Ibis Server). |
| `projects/idxfin/wren_project.yml` | Tells Wren Engine which DuckDB file to read and that it must be read-only. |
| `projects/idxfin/models/*.yml` | The MDL (Modeling Definition Language) semantic models — one YAML file per Gold view, describing columns, descriptions, and calculated fields (ROE, D/E ratio, etc.). See Section 9.3. |
| `projects/idxfin/relationships.yml` | How the semantic models join to each other (e.g. financial_summary ↔ market_valuation on issuer + period). |

| | |
|---|---|
| `projects/idxfin/knowledge/accounting_rules.yaml` | Plain-English domain rules injected into the AI's context (e.g. "never sum USD and IDR companies together"). |
| `projects/idxfin/knowledge/golden_queries.ya` | Hand-verified example question → SQL pairs, used both as few-shot examples and as regression tests. |

# 4. The Data Journey: From PDF to Database, Step by Step

This section walks through **exactly** what happens when you run:

```
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf
```

...tracing the real function calls in `src/pipeline.py`.

## 4.1 Step 1 — Open and fingerprint the PDF (`PDFDocument`)

`src/parser/pdf_loader.py` uses **PyMuPDF** (the `fitz` library) to open the file. Two important things happen immediately:

- The raw file bytes are hashed with SHA-256 to produce `doc_id`. This is the document's permanent fingerprint — if you feed the exact same PDF in twice, it gets the exact same ID, which is what makes re-runs safe (idempotent) instead of creating duplicate rows.
- Every page's text and text-block bounding boxes are extracted up front into a list of `PDFPage` objects, so later stages never need to re-open the PDF file.

Then a batch of regex-based "sniffing" runs over the first ~20 pages to guess:

| Field | How it's detected | Example |
|---|---|---|
| `doc_type` | Filename keywords first ("AnnualReport", "FinancialStatement"), then content heuristics. | `fs_annual`, `fs_interim`, `annual_report` |
| `ticker` | Regex for "Kode Emiten:", else the 4 uppercase letters in the parent folder name or filename. | `NCKL` |
| `currency` | Looks for "Dolar Amerika/USD" vs "Rupiah/IDR" phrases. | `IDR` or `USD` |
| `scale` | Looks for "Dalam jutaan" (millions), "ribuan" (thousands), "miliaran" (billions), else full units. | `1,000,000` |
| `number_locale` | Counts occurrences of the English pattern `1,234,567` vs. the Indonesian pattern `1.234.567` and picks the majority. | `en` or `id` |
| `fiscal_year` / `fiscal_period` | Regex for "31 Desember 20XX" (FY), "30 September" (9M), "30 Juni" (H1), "31 Maret" (Q1). | `FY 2025` |

> **Watch out:** this is regex-based, so it can be wrong on unusual phrasing (e.g. "Angka disajikan dalam triliun Rupiah" — trillions — is not one of the patterns it looks for, and would silently default to full units). The codebase's own design docs (`docs/LLM_FIRST_IMPLEMENTATION_PLAN.md`) flag exactly this weakness and propose replacing it with a single small LLM call over the cover pages (there's already a working `extract_metadata_with_llm()` method on `PDFDocument` for this, it just isn't wired into the default CLI path yet). If a filing's numbers look off by a factor of 1,000 or 1,000,000, this scale-sniffing step is the first place to check.

## 4.2 Step 2 — Cut the document into sections (`Sectionizer`)

Indonesian financial statements are built on an official IDX taxonomy where every statement and note is tagged with a 7-digit code in square brackets, like `[1210000]`. The `Sectionizer` scans every page for the pattern `\[(\d{7})\]\s*(title)` and turns each match into a `SectionSpan` (an ID, a page range, and a "section_type").

The mapping from code to meaning is based on the **second digit** of the 6-digit tail (this is the actual IDX XBRL taxonomy convention):

| Second digit | Meaning | Example code |
|---|---|---|
| `2` | Balance Sheet / Statement of Financial Position | `[1210000]` |
| `3` | Income Statement / P&L and OCI | `[1311000]` |
| `4` | Statement of Changes in Equity | `[1410000]` |
| `5` | Statement of Cash Flows | `[1510000]` |
| `6...` | Notes to the financial statements (further sub-routed by the next digits: PPE, receivables, borrowings, bank loans/deposits, etc.) | `[1611100]`, `[1691100]` |

For a 400+ page **Annual Report**, there are no neat bracketed codes — instead, the sectionizer scans page headers for known Indonesian/English chapter titles ("Ikhtisar Keuangan" = Financial Highlights, "Tinjauan Kinerja Operasional" = Operational Performance Review, "Profil Direksi" = Board Profiles, etc.) using a library of regex patterns. There is also an optional, smarter path: if an LLM client is supplied, the sectionizer first tries to find and parse the document's own **Table of Contents** pages with an LLM (`_sectionize_ar_with_llm`) to get exact chapter boundaries directly from the document's own index, which is far more robust for creatively-titled chapters than scanning 400 pages of regexes.

## 4.3 Step 3 — Route each section to a specialized extractor

`ExtractionRouter.route_and_extract()` loops over every detected section and dispatches based on `section_type`:

| section_type | Goes to | Produces |
|---|---|---|
| `lk.statement`, `lk.note.table`, `lk.note.borrowings` | `StatementExtractor` | A `StatementExtraction` (line items × periods × cells) |
| `ar.mdna.kpis`, `sr.esg_metrics`, `deck.kpis` | `KPIExtractor` | A list of `KPIFact` (production volumes, sales volumes, etc.) |
| `ar.governance.officers` | `GovernanceExtractor` | A list of `OfficerPosition` (board members) |
| `ar.mdna.outlook`, `deck.guidance` | `AssertionsExtractor` | A list of `ForwardGuidance` (management targets) |
| `lk.auditor_report` | `AuditExtractor` | One `AuditReport` (audit firm, opinion, KAM) |

Each extractor follows the same recipe: pull the raw page text for the section's page range, build a prompt that includes candidate concept IDs, call the LLM demanding a specific Pydantic schema back, and return the parsed object. If any single section throws an exception, the router logs it and *keeps going* — one bad note shouldn't sink the whole filing (this is why the README calls it "fault-tolerant": you still get all the other statements committed and validated).

## 4.4 Step 4 — The LLM call itself: structured output + candidate injection

This is the heart of the "LLM finds things, code decides the truth" philosophy. Two techniques make the LLM's output safe to trust structurally (not necessarily numerically — that's what validation is for):

### 4.4.1 Pydantic schema enforcement

`OpenRouterClient.generate_structured()` (in `src/llm/client.py`) takes any Pydantic model class (e.g. `StatementExtraction`) and:

1. Converts it to a JSON Schema and pastes that schema into the system prompt with the instruction "you MUST output valid JSON matching this schema".
2. Requests the response in JSON mode.
3. Parses the raw text with `response_model.model_validate_json(cleaned)`. If that fails validation, it does **one repair round-trip**: it sends the bad output back to the LLM along with the exact Pydantic validation error message and asks it to fix the JSON.

This means a badly-shaped LLM response (missing field, wrong type, extra text before the JSON) either gets auto-repaired or raises a clear Python exception — it can never silently corrupt the database, because the corrupted shape simply won't parse.

### 4.4.2 Candidate concept injection (the anti-hallucination trick for taxonomy)

Instead of asking the LLM "what's the standard concept ID for this line item?" with a totally open answer space (which invites the LLM to invent plausible-but-fake IDs like `fs.bs.total_asset` with a typo), the prompt **injects the actual list of valid concept IDs** for that statement type (drawn from `schema_design/taxonomy_seed.yaml` via `ConceptMapper.get_candidates_for_statement()`) and instructs the model: "assign `concept_candidates` ONLY from the candidate list provided". The Pydantic field itself is also typed as a bounded list (max 5 items). This narrows an open-vocabulary generation problem into a closed-set classification problem, which LLMs are dramatically more reliable at.

## 4.5 Step 5 — Normalize: turn LLM output into trustworthy numbers

`Normalizer.normalize_statement()` (in `src/normalization/normalizer.py`) walks every line item × cell the LLM found and, for each one:

1. **Maps the concept.** It tries, in order: (a) exact dictionary lookup (`ConceptMapper.map_label`, ~250 hand-built mappings plus 148 taxonomy labels, normalized to lowercase with punctuation stripped); (b) fuzzy token-overlap matching (Jaccard similarity > 0.80 on 3+ word phrases); (c) whatever candidate the LLM itself already picked; (d) as a last resort, an LLM few-shot resolver that is shown ~15-25 known examples and asked to classify the new label, then **persists** the answer into `idxfin_ref.label_mapping` so the exact same label is an instant dictionary hit next time (see the callout below).
2. **Parses the number, deterministically.** `parse_reported_number()` (in `src/normalization/number_parser.py`) handles: dash/"nihil"/"nil" → zero; parentheses or a leading minus → negative; locale-aware separators (Indonesian `1.234.567,89` vs English `1,234,567.89`); then strips currency symbols and percent signs. This function is pure, has no dependency on any LLM, and is directly unit-tested (see `tests/test_number_parser.py`).
3. **Applies scale.** `apply_scale(value, scale)` multiplies by the document's detected scale (1, 1,000, 1,000,000, or 1,000,000,000) so every stored value is in full nominal units, comparable across companies regardless of how each one chose to print its numbers.
4. **Builds a stable fact ID.** `make_fact_id()` hashes `(doc_id, concept_or_label, period_end, dim_key, scenario, raw_text)` with SHA-256 and truncates to 32 hex characters. Because this hash is deterministic, re-running extraction on the same PDF produces the same fact IDs, and the database write uses `INSERT OR REPLACE` — so re-runs upsert cleanly instead of duplicating rows.

> **Why the mapper persists new labels to the database:** this turns the concept mapper into a system that *gets cheaper and more accurate over time* without retraining anything. The first time the pipeline sees "Pendapatan Jasa Boga Penerbangan" (in-flight catering revenue, from CASS an aviation caterer) it costs one LLM call; every subsequent filing that uses that exact label anywhere resolves in O(1) time with zero tokens spent, because it's now a plain dictionary lookup backed by `idxfin_ref.label_mapping`.

## 4.6 Step 6 — Validate: does the accounting actually add up?

`ValidationEngine.validate_facts()` runs two categories of checks, described fully with all rules in Section 5.4. In short:

- **Grounding (hard rule, runs first, on every fact):** the raw text the LLM claimed it saw must actually appear on the cited PDF page (whitespace-insensitive), checked with a ±1 page tolerance for tables that straddle a page break. This is the last line of defense against a hallucinated number — if the exact string "32,824,207" isn't really printed on page 4 (or 3 or 5), the fact fails validation.
- **Accounting identities (mostly hard rules):** balance sheet identity, current/non-current split, gross profit, net profit, cash flow sections. Each one computes a left-hand-side and right-hand-side value from already-normalized facts and checks the difference is within a rounding tolerance (3× the document's own reporting scale, so a company reporting in millions gets a few million rupiah of legitimate rounding slack).

When a hard rule *fails*, before giving up, the engine can call `reconcile_variance_with_llm()`: it gathers the exact PDF pages behind the implicated facts, asks an LLM "is there a legitimate accounting reason for this gap (e.g. PSAK 101 Sharia funds, discontinued operations, FX effects)?", and if the LLM names a specific reconciling value, Python *independently re-computes* whether adding/subtracting that value actually closes the gap to zero before accepting it. The LLM never gets to just declare "trust me, it balances" — its suggestion is treated as a hypothesis that deterministic arithmetic must confirm.

## 4.7 Step 7 — Repair loop and review tasks

Any hard rule that still fails after the LLM reconciliation attempt gets turned into a `ReviewTask` (`src/validation/repair.py`) with a priority score (0.8 for a grounding failure, 0.9 for an equation failure) — these are written to `idxfin_silver.review_task` for a human analyst to look at later. The facts themselves are *not* deleted or hidden; they're kept with their full trail so nothing is silently lost.

## 4.8 Step 8 — Stage to Parquet, then commit to DuckDB

Everything produced so far (facts, officers, guidance, audit reports, validation results, review tasks, plus the document's own bronze metadata) is written as a set of Parquet files under `data/staging/pending/<run_id>/` by `ParquetStagingStore`. Then, unless running in `staging_only` mode (which is what Celery extraction workers use), `DatabaseStore.ingest_from_staging()` loads every Parquet file into its matching DuckDB table with `INSERT OR REPLACE ... SELECT ... FROM read_parquet(...)`, all inside one `FileLock`-protected block, and finally re-creates every Gold view. The staging run then moves from the `pending/` folder to `completed/`.

# 5. The Database Schema, Explained Table by Table

## 5.1 Why one big "fact" table instead of one table per industry

The design docs are explicit about this decision (`schema_design/DESIGN.md` §1.1): a naive design would create tables like `bank_balance_sheet` or `mining_operational_report`. That breaks the moment an insurer shows up, or a mining company adds a smelter segment, because you'd need a schema migration for every new business shape. Instead, everything is stored as one universal **fact** shaped like an XBRL data point:

```
(issuer, concept, period, dimensions, scenario) -> (value, unit, provenance, assurance)
```

In plain terms: *who* reported *what metric*, *for what time period and slice* (e.g. "Nickel segment" or "Q3 2025"), under *what scenario* (actual vs. guidance) — and then the actual number, its unit, where it came from, and how trustworthy it is (audited vs. unaudited). Adding a brand-new industry (say, insurance) means adding new *rows* to the concept dictionary, never a new *column* or table.

## 5.2 Reference layer — `idxfin_ref`

| Table | Key columns | What it's for |
|---|---|---|
| `concept` | `concept_id` (PK, e.g. `fs.bs.total_assets`), `domain`, `statement`, `label_en`, `label_id`, `data_type`, `balance` (dr/cr), `period_type` (instant/duration) | The 148-row taxonomy "dictionary" seeded from `taxonomy_seed.yaml` at startup. |
| `label_mapping` | `label_raw` (PK), `concept_id`, `mapping_method`, `confidence` | The self-growing memory of "we've seen this exact raw label before and know what it means" — written to every time the LLM resolver classifies a brand-new label. |

## 5.3 Bronze layer — `idxfin_bronze`

| Table | Key columns | What it's for |
|---|---|---|
| `document` | `doc_id` (PK, SHA-256 of the file bytes), `issuer_id`, `doc_type`, `fiscal_year`, `fiscal_period`, `period_end`, `currency`, `scale`, `number_locale`, `page_count` | One row per ingested PDF — the audit trail of "what did we read, and what did we think it was". |

| | | |
|---|---|---|
| `page` | `doc_id`, `page_no`, `text` | Raw per-page text, so any downstream tool (like the grounding checker) can re-read a page without re-opening the original PDF file. |

## 5.4 Silver layer — `idxfin_silver` (the workhorse)

### 5.4.1 `fact` — the universal fact table

This is the single most important table in the whole system. Every number extracted from every statement, note, and KPI disclosure lands here as one row.

| Column group | Columns | Meaning |
|---|---|---|
| Identity | `fact_id` (PK) | Deterministic SHA-256 hash — same input always produces the same ID, so re-runs upsert safely. |
| What | `concept_id`, `label_raw`, `label_raw_alt`, `note_ref`, `mapping_method`, `mapping_confidence` | The standardized meaning of the number, plus the original as-printed label(s) in both languages, and how confident/how the mapping was made. |
| When | `period_type` (instant/duration), `period_start`, `period_end`, `fiscal_year`, `fiscal_period`, `duration_months`, `is_cumulative_ytd` | Balance-sheet items are "instant" (a snapshot on one date); income/cash-flow items are "duration" (a span of time). Interim filings are cumulative year-to-date, which matters a lot for quarter math (see §5.4.3). |
| Context | `dim_key`, `scenario`, `consolidation_scope`, `basis` | `dim_key` is a sorted string of "axis=member" pairs (e.g. segment or currency slices) — empty string means "the consolidated total, no slicing". |
| Value | `value` (scaled Decimal), `raw_text` (verbatim as printed), `currency`, `scale_as_reported` | `value` is always the fully-scaled, comparable number; `raw_text` keeps the original printed string forever for audit purposes. |

| | | |
|---|---|---|
| Trust | `assurance` (audited/reviewed/unaudited), `is_restated_column`, `is_comparative`, `review_status`, `knowledge_ts` | Not every number in a filing has the same trust level — the current column of a Q1 filing is unaudited, but its prior-year comparative column is audited. `knowledge_ts` is when the fact was *learned*, separate from the period it describes (this enables point-in-time, look-ahead-bias-free queries). |
| Lineage | `lineage_page`, `lineage_extractor`, `lineage_method`, `lineage_model`, `lineage_run_id`, `lineage_confidence` | The full "how do we know this" trail — which PDF page, which extractor, which LLM, which pipeline run. |

### 5.4.2 Other Silver tables

| Table | What it holds |
|---|---|
| `officer_position` | Board of Directors / Board of Commissioners profiles: name, role, independence flag, source page. |
| `audit_report` | Independent Auditor's Report metadata: audit firm (KAP), network (Big 4 or local), signing partner, opinion type, Key Audit Matters. |
| `forward_guidance` | Management's own stated targets/outlook from the MD&A section (point or range values, target period). |
| `validation_result` | Every single accounting-rule check that was ever run: rule ID, pass/fail, LHS, RHS, diff, tolerance, message. This is your full audit log of the validation engine. |
| `review_task` | The queue of unresolved validation failures waiting on a human, with a priority score. |

### 5.4.3 Why "cumulative quarters" need special handling

Indonesian interim filings report **year-to-date** numbers, not standalone quarters: a "9M" (nine-month) filing's income statement shows 9 months of revenue, not just Q3 alone. There is no such thing as a standalone "Q4 filing" — Q4 only ever exists as `FY − 9M`. The Gold view `fact_discrete_quarter` (Section 5.6) does this subtraction for you, but it deliberately **refuses** to compute Q2 if Q1 is missing, and it flags any derived Q4 as `absorbs_year_end_adjustments = true` because year-end audit adjustments and bonus true-ups all land in that leftover quarter.

## 5.5 Market layer — `idxfin_market`

| Table | What it holds |
|---|---|
| `stock_daily` | 4 years of daily OHLCV (open/high/low/close/volume) bars for IDX-listed tickers, pulled via `yfinance`. |
| `commodity_daily` | Gold (`GC=F`), WTI & Brent crude oil, Henry Hub natural gas, and an ammonia "proxy basket" (`ESSA.JK`, `CF`, `UAN`, plus a synthetic `NH3=F` physical spot benchmark) — because raw physical ammonia doesn't trade on a public exchange, so a triangulated proxy is used instead. |
| `fx_daily` | Daily USD/IDR exchange rates, used to bridge USD-reporting issuers (ADRO, ARCI, BREN, ABMM, ESSA) with IDR-reporting ones. |

This is deliberately kept in its own schema, separate from `idxfin_silver`, because it comes from a completely different trust source (a public market data feed, not an audited filing) and has its own refresh cadence.

## 5.6 Gold layer — `idxfin_gold` (what people actually query)

| View | What it does |
|---|---|
| `fact_latest` | For every (issuer, concept, period, dimension) combination, picks the single "best" fact using a window function ordered by assurance level (audited beats unaudited) then by how recently it was learned. This is how restatements are handled — nothing is overwritten, the view just always shows you the best current answer. |
| `fact_first_reported` | The mirror image: picks the *earliest-known* version of each fact. Useful for "what did the market actually see on day one" analysis, e.g. earnings surprises, before comparing to later restated figures. |
| `fact_discrete_quarter` | Derives standalone Q2/Q3/Q4 numbers by subtracting cumulative periods (H1 – Q1, 9M – H1, FY – 9M). |
| `financial_summary` | The main "one row per company per period" pivot: total assets, liabilities, equity, revenue, gross profit, net profit, operating cash flow, capex, plus banking-specific fields (loans, deposits). Uses layered `COALESCE(...)` chains so a bank's "interest income" and a retailer's "revenue" both land in the same `revenue` column. |

| | |
|---|---|
| `financial_summary_provenance` | The exact same shape as `financial_summary`, but every number is replaced by its source PDF page citation (e.g. `"p.18"`) — built so any answer can be instantly double-checked against the original document. |
| `market_valuation` | Joins `financial_summary` to `stock_daily`, finding the closing price closest to each filing's period-end date (within ±5 days) and the latest available price, to compute price performance since the filing. |
| `commodity_trend_monthly` | Monthly aggregation of commodity prices: average/min/max close, month-open, month-end, and month-over-month percentage return. |
| `operational_kpi` | Filters `fact_latest` down to just the operational metrics (concept IDs starting with `op.` or `reg.`) — production volumes, capacity, regulatory ratios. |
| `fx_rate_daily` | Daily FX close rates with rolling monthly and yearly average rates, used to translate USD figures into IDR (or vice versa) for cross-currency comparisons. |

**How to explore the schema yourself:** open a terminal and run `duckdb data/idxfin.duckdb`, then inside the DuckDB shell: `.mode box`, `SHOW TABLES FROM idxfin_gold;`, `DESCRIBE idxfin_silver.fact;`. The README's "Data Inspection Playbook" has nine ready-to-run SQL recipes (balance sheet reconciliation, raw-cell audit, comparative time series, governance, validation audit log, cross-asset valuation, commodity trends, USD→IDR conversion, ammonia vs. natural gas) — copy any of them straight into the DuckDB CLI or a Python script.

# 6. The Command-Line Interface (CLI) Reference

Everything runs through one entry point: `python3 -m src.cli <command> [options]`. All commands are defined with `argparse` sub-parsers in `src/cli.py`.

## 6.1 `inspect` — free, offline document profiling

```
python3 -m src.cli inspect examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf
```

Opens the PDF, prints the sniffed metadata (ticker, doc type, period, currency, scale, locale) and every detected section span with its page range — **without calling any LLM**. Always run this first on a new filing before spending money on `extract`, so you can sanity-check that the sectionizer found what you expect.

## 6.2 `extract` — run the full pipeline

```
# Every section in the document
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf

# Only specific sections (comma-separated section IDs or types)
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf \
 --sections lk.1210000,lk.1311000,lk.1410000,lk.1510000

# One single section, useful while debugging a prompt
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf \
 --sections lk.1311000.p18

# Pick a specific model for this run
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf --model
anthropic/claude-3.5-sonnet

# Enqueue asynchronously to Celery/Redis instead of running in-process
python3 -m src.cli extract examples/nckl/FinancialStatement-2025-Tahunan-NCKL.pdf --async
```

Without `--async`, this runs `ExtractionPipeline.run()` synchronously in your current process and prints a full summary (facts extracted, hard-rule pass rate, review tasks created) plus every validation rule's pass/fail status.

## 6.3 `status <task_id>` — poll an async job

Looks up a Celery task by ID (from an `--async` extract or an API call) and prints its state: `PENDING` → `STARTED` → `PROGRESS` → `SUCCESS`/`FAILURE`.

## 6.4 `gold [--ticker TICKER]` — query the analytics layer

Prints `idxfin_gold.financial_summary` (optionally filtered to one ticker) as a nicely formatted table, followed by the matching rows from `financial_summary_provenance` so you can see the page citation for every number right next to it.

## 6.5 `market` — sync stock/commodity/FX prices

```
python3 -m src.cli market --summary # print current coverage
python3 -m src.cli market --tickers BBCA,ADRO,NCKL,ARCI --period 4y # sync specific
stocks
python3 -m src.cli market --commodities --period 4y # sync Gold/Oil/Ammonia proxies
python3 -m src.cli market --fx --period 4y # sync USD/IDR
python3 -m src.cli market --all-tickers --commodities --fx # sync everything known to the
DB
```

## 6.6 `api [--host] [--port] [--reload]`

Starts the FastAPI gateway with `uvicorn` (default `0.0.0.0:8000`). See Section 8 for the routes.

## 6.7 `worker [-Q queue] [-c concurrency]`

Launches a Celery worker process pointed at either the `extraction` queue (default concurrency 6) or the `duckdb_writer` queue (must stay at concurrency 1 — see Section 7).

## 6.8 `reindex-staging` — disaster recovery

Walks every run folder under `data/staging/pending/` and `data/staging/completed/` and re-ingests each one into DuckDB. Use this if the DuckDB file is ever lost, corrupted, or you want to rebuild a fresh database without spending a single dollar on LLM calls again — the Parquet files already contain everything.

# 7. Going Distributed: Redis, Celery, and the Staging Lake

## 7.1 Why two separate queues, not one

`src/celery_app.py` declares exactly two Celery queues, and they exist for opposite reasons:

| Queue | Concurrency | Why |
|---|---|---|
| `extraction` | 6 (configurable via `CELERY_WORKER_CONCURRENCY`) | LLM calls are slow (network I/O) and independent of each other — more parallelism here means more filings processed per hour. |
| `duckdb_writer` | **Always 1** | DuckDB physically cannot have two writers at once — this queue exists purely to serialize commits, on purpose. |

## 7.2 Backpressure settings, and why they matter

| Setting | Value | What it prevents |
|---|---|---|
| `worker_prefetch_multiplier` | 1 | Without this, a Celery worker will greedily grab several tasks into local memory ahead of time. Setting it to 1 means a task only leaves the Redis queue the moment a worker is actually free to start it — so the queue depth you see in Redis reflects real, current backlog, not tasks silently hoarded by a busy worker. |
| `task_acks_late` | True | The task is only marked "done" in Redis *after* it finishes successfully. If a worker process crashes mid-extraction (e.g. out of memory on a huge annual report), the unfinished task is automatically re-queued and picked up by another worker instead of being silently lost. |

## 7.3 The lifecycle of one async job

1. A client calls `POST /api/v1/extract/fs` (or the CLI with `--async`).
2. FastAPI enqueues `extract_document_task` onto the `extraction` queue and immediately returns `202 Accepted` with a `task_id` and a `poll_url`.
3. One of the 6 extraction workers picks it up, runs `ExtractionPipeline(staging_only=True).run(...)` — note `staging_only=True` means this worker **never opens the DuckDB file at all**, it only writes Parquet.
4. On success, it automatically enqueues `ingest_staging_task(run_id)` onto the `duckdb_writer` queue.
5. The single `duckdb_writer` worker loads the Parquet files, rebuilds Gold views, and moves the run folder from `pending/` to `completed/`.
6. The client polls `GET /api/v1/tasks/{task_id}` until `state == "SUCCESS"`.

## 7.4 Docker Compose topology

`docker-compose.yml` defines four services: `redis` (the broker/result-backend), `api` (FastAPI gateway), `worker-extraction` (concurrency 6), and `worker-duckdb-writer` (concurrency 1, hard-coded with `-c 1` so it can never accidentally be scaled up). All four share the same `./data` volume so they see the same DuckDB file and staging folders.

# 8. The FastAPI Gateway

Defined across `src/api/app.py` (app + health check), `routes.py` (endpoints), and `schemas.py` (request/response models).

| Method & Path | What it does |
|---|---|
| `POST /api/v1/extract/fs` | Enqueue a Financial Statement (LK) for extraction, doc_type forced to `fs_annual`. |
| `POST /api/v1/extract/ar` | Enqueue an Annual Report, doc_type forced to `annual_report`. |
| `POST /api/v1/extract` | Universal endpoint; the pipeline auto-detects the doc type itself. |
| `GET /api/v1/tasks/{task_id}` | Poll a Celery task's status/progress/result/error via `AsyncResult`. |
| `GET /api/v1/tasks` | Inspect worker pool health: Redis connectivity, registered workers, active/scheduled/reserved task counts, per-queue depth. |
| `GET /api/v1/gold/summary?ticker=NCKL` | Read-only query straight into `idxfin_gold.financial_summary`, returned as JSON. |
| `POST /api/v1/ingest/{run_id}` | Manually re-trigger DuckDB ingestion for an already-staged Parquet run (useful for retries). |

Notice the request path resolution helper `_resolve_file_path()` in `routes.py`: it tries the path as given, then relative to the project root, then looks for known anchor folders (`examples`, `complete-ticker`, `data`) inside the given path — this is what lets the same request body work whether you're calling the API from the host machine or from inside a Docker container with a different absolute path.

# 9. Wren AI: Talking to the Data in Plain English

## 9.1 What problem is Wren AI actually solving?

By the time you reach the Gold layer, you have clean, validated, well-modeled data — but an analyst still has to know SQL and the exact table/column names to use it. "GenBI" (Generative Business Intelligence) means: let a person ask a question in plain English or Bahasa Indonesia, and have the system write and run the correct SQL for them. The naive version of this — just handing an LLM your raw database schema and asking it to write SQL ("naked text-to-SQL") — fails in predictable ways on this specific dataset:

| Failure mode of naive text-to-SQL | How this project prevents it |
|---|---|
| Invents column/table names that don't exist | The LLM only ever sees a curated **semantic model** (MDL), not the raw warehouse — described in §9.3. |
| Ignores PSAK 101 Sharia rules, thinks the balance sheet doesn't balance | The knowledge base explicitly documents the `Assets = Liabilities + Equity + Syirkah` identity as a rule the model must apply. |
| Adds a USD company's numbers to an IDR company's numbers | An explicit "currency isolation" rule is injected into every query's context. |
| No way to check if an answer is trustworthy | Every model is linked to a provenance view (page citations), and the system prompt instructs the agent to cite the source page whenever asked to verify a number. |

## 9.2 Two ways to use it

| Option | What it is | When to use it |
|---|---|---|
| A — `wren/chat_agent.py` | A pure-Python terminal chatbot, no Docker required. See Section 10 for the full internals. | Fastest way to ask a question right now, or to demo the concept. |
| B — Full Docker stack (`wren/docker-compose.wren.yml`) | 5 microservices: `wren-ui` (web chat + visual modeler, port 3000), `wren-engine` (Rust/Apache DataFusion semantic SQL planner, port 8080), `wren-ai-service` (FastAPI + LiteLLM orchestration, port 5555), `qdrant` (vector database for schema/few-shot search, port 6333), `ibis-server` (query execution, port 8000). | A real multi-user web UI, visual data exploration, dashboards, and a full conversational history. |

## 9.3 MDL: the Modeling Definition Language

Every Gold view gets its own YAML "model" file under `wren/projects/idxfin/models/`. Each one declares, in plain language, what each column means and what calculated metrics can be derived from it. For example, `financial_summary.yaml` defines `calculated_fields` like:

```
- name: return_on_equity_pct
 expression: "ROUND((net_profit / NULLIF(total_equity, 0)) * 100, 2)"
 description: "Return on Equity (ROE) percentage."

- name: loan_to_deposit_ratio_pct
 expression: "ROUND((bank_loans / NULLIF(bank_deposits, 0)) * 100, 2)"
 description: "Loan to Deposit Ratio (LDR) for commercial banks."
```

This means the AI never has to correctly write the ROE formula from scratch inside a generated SQL query (a place where it could easily get the formula subtly wrong, e.g. dividing by total assets instead of equity) — it just needs to recognize that the user is asking for ROE and reference the calculated field the engineers already wrote and reviewed once. `relationships.yml` then declares how models join to each other (e.g. `financial_summary` to `market_valuation` on `issuer_id + period_end`) so the AI doesn't have to guess join keys either.

## 9.4 The knowledge folder: rules and golden examples

`knowledge/accounting_rules.yaml` encodes plain-English domain rules as short paragraphs (e.g. "NEVER add USD and IDR directly without grouping by currency or converting via exchange rate"), and `knowledge/golden_queries.yaml` holds hand-verified question → SQL pairs. These golden queries serve two jobs at once: they're few-shot examples fed to the AI, *and* they're executed directly as a regression test in `tests/test_wren_semantic_contract.py::test_golden_queries_execution` — so if a Gold view's shape ever changes and breaks a known-good query, the test suite catches it immediately, before an end user ever notices a wrong chatbot answer.

# 10. Inside chat_agent.py: How the Chat Agent Really Works

This is a deep, line-by-line-style walkthrough of `wren/chat_agent.py` because the prompt specifically asked "how does chat_agents work" — this is the file that answers that.

## 10.1 The moving parts, at a glance

| Piece | Library | Job |
|---|---|---|
| LLM | `langchain_openai.ChatOpenAI` | Talks to an OpenAI-compatible chat model. |
| Agent loop | `langgraph.prebuilt.create_re` | Implements the "ReAct" pattern: the model can think, decide to call a tool, observe the tool's result, and repeat, until it's ready to answer in plain text. |
| Tools | `wren_langchain.WrenToolkit` | Wraps Wren's semantic engine as LangChain tools the agent can call (e.g. run a semantic SQL query against the MDL models). |
| Terminal UI | `rich` | Pretty console output: colored panels for tool calls, syntax-highlighted SQL, Markdown-rendered answers. |

> **A real gotcha worth knowing before you run this:** `langchain_openai`, `langgraph`, `wren_langchain`, and `rich` are **not** listed in the project's root `requirements.txt` (that file only covers the core extraction pipeline: pydantic, PyMuPDF, duckdb, celery, fastapi, etc.). If you try to run `python wren/chat_agent.py` fresh and get `ModuleNotFoundError`, that's why — you need to separately `pip install langchain-openai langgraph wren-langchain rich python-dotenv` (or check whether a dedicated `wren/requirements.txt` exists in your checkout / is documented in the Wren AI tutorial before assuming it's missing entirely).

## 10.2 Startup: building the agent (`create_agent`)

1. **Load environment variables twice:** first the project root `.env`, then `wren/.env.wren` as a fallback/override — so Wren-specific settings can live separately from the core pipeline's settings.
2. **Build the toolkit:** `WrenToolkit.from_project(project_path)` points at `wren/projects/idxfin` and loads the MDL models/relationships, exposing them as callable tools (`toolkit.get_tools()`).

3. **Pick a model, with a cascading fallback:** the code tries, in order, the primary model (`DEFAULT_MODEL` env var, or `gpt-5.6-luna` if unset), then the fallback model (`FALLBACK_MODEL`, or `gpt-5.4-mini`), then hard-coded `gpt-4o`, then `gpt-4o-mini` — stopping at the first one that initializes successfully. Note `reasoning_effort="none"` is passed on every attempt; the code comment explains this is required specifically to let a "gpt-5.x"-style reasoning model be bound to tools in Chat Completions mode.

4. **Combine the system prompt:** the final instructions given to the model are `toolkit.system_prompt()` (Wren's own description of the available tools/models) concatenated with this project's own `PSAK_SYSTEM_GROUNDING` block.

5. **Wire it into a ReAct agent:** `create_react_agent(llm, tools=tools, prompt=combined_prompt)` from LangGraph returns a ready-to-invoke agent graph.

## 10.3 The domain grounding prompt (`PSAK_SYSTEM_GROUNDING`)

This constant is essentially the "operating manual" the model is forced to read before answering anything. In plain English, it tells the model:

- Tickers are 4-letter IDX codes (ESSA, BBCA, NCKL, ADRO...).
- How the PSAK 101 Sharia balance sheet identity works, and which company (BBCA, via its sharia subsidiary) needs it.
- Never mix IDR and USD figures without going through the `fx_rate` model first, and use the point-in-time rate for balance-sheet items but the yearly average rate for income-statement items.
- Banks (BBCA, BBSI) don't have a current/non-current split — don't query those fields for them, use `bank_loans`/`bank_deposits`/`loan_to_deposit_ratio_pct` instead.
- Which commodity ticker symbols mean what (`GC=F` = gold, `NH3=F` = physical ammonia benchmark, `NG=F` = natural gas feedstock, etc.).
- Bilingual disclosure terms to search for both languages at once (e.g. Ammonia = "Amoniak"/"Amonia"; Coal = "Batubara").
- Output formatting rules (thousands separators, explicit currency codes, 2-decimal percentages, Markdown tables).

This is exactly the same idea as the "knowledge" YAML files from Section 9.4, just delivered as one big prompt block instead of separate MDL entries — because this file talks to Wren's engine tools directly rather than through the full Docker AI service.

## 10.4 Running one query (`run_query`)

```
def run_query(agent, toolkit, question, explain=False):
   messages = [HumanMessage(content=question)]
   response = agent.invoke({"messages": messages}, {"recursion_limit": 15})
   if explain:
       print_execution_trace(response["messages"])
   return response["messages"][-1].content
```

The user's question becomes a single `HumanMessage`. `agent.invoke(...)` runs the full ReAct loop: the model reads the question, decides it needs data, emits a tool call (typically `wren_query` with a semantic SQL string), LangGraph executes that tool and feeds the result back to the model as an observation, and this can repeat multiple times (up to `recursion_limit=15` steps, a safety cap against infinite tool-calling loops) before the model finally writes a plain-text final answer. The very last message in the returned list is that final answer.

## 10.5 Watching the agent think (`--explain` / `print_execution_trace`)

Every intermediate message that contains `tool_calls` gets rendered as a `rich` panel. If the tool being called is `wren_query`, the SQL argument is pulled out and displayed with SQL syntax highlighting inside its own panel, subtitled with the row `limit` that was used. This is invaluable for debugging: run any question with `--explain` and you can see the *exact* semantic SQL the model generated, before it gets translated further down into real DuckDB SQL by Wren Engine.

## 10.6 The two ways to talk to it

```
# One-shot mode: ask one question and exit
python wren/chat_agent.py -q "Berapa laba bersih dan pendapatan ESSA tahun 2024 dan
2025?"

# Interactive REPL: keep a conversation open until you type exit/quit/q
python wren/chat_agent.py

# See the generated SQL and tool calls for any question
python wren/chat_agent.py -q "Verify BBCA's balance sheet reconciliation" --explain
```

In REPL mode (the `repl()` function), each turn is handled independently — note that `run_query` builds a brand-new one-message list every call rather than accumulating full chat history across turns, so this agent currently behaves more like "ask a fresh, fully self-contained question each time" than "have a long multi-turn memory conversation". This is worth knowing if a follow-up question like "and what about last year?" doesn't get the context you expect.

# 11. How to Debug This Project: A Practical Playbook

## 11.1 First move: isolate which layer is broken

Because the pipeline is a strict pipe (parse → sectionize → extract → normalize → validate → store), the fastest way to debug anything is to run each stage in isolation rather than the whole `extract` command:

```
# Stage 1+2 only, zero cost, zero LLM calls -- confirms sectionizer found the right pages
python3 -m src.cli inspect path/to/file.pdf

# Stage 3+ on ONE section only -- cheapest way to test/iterate on a single prompt
python3 -m src.cli extract path/to/file.pdf --sections lk.1311000.p18
```

## 11.2 "My numbers are off by 1,000 / 1,000,000"

This is almost always a **scale detection** bug (Section 4.1). Run `inspect` and check the printed `Scale:` line. If it says `1 (Satuan penuh / As reported)` but the filing actually says "Dalam jutaan Rupiah" using unusual phrasing the regex doesn't recognize, that's your bug — the fix is either to extend the phrase list in `PDFDocument._detect_filing_metadata()`, or to wire in the already-implemented but not-yet-default `extract_metadata_with_llm()` path for that filing.

## 11.3 "A specific number looks hallucinated / wrong"

1. Find the fact in `idxfin_silver.fact` and read its `raw_text` and `lineage_page` columns.
2. Open the source PDF to that exact page and visually confirm the string is really there.
3. Check `idxfin_silver.validation_result WHERE rule_id = 'grounding' AND passed = false` — if grounding already failed for this fact, it should already be sitting in `review_task`, and you can see exactly why (the raw text didn't match, possibly a page-boundary table).
4. If grounding passed but the number still looks wrong, the issue is more likely a **concept mapping** mistake (right number, wrong meaning/label) — check `mapping_method` and `mapping_confidence` on that fact; anything from `extension_fallback` (confidence 0.5) deserves a second look.

## 11.4 "The balance sheet doesn't balance"

Query `idxfin_silver.validation_result WHERE rule_id = 'bs_identity' AND passed = false`. The row's `message` field literally spells out LHS, RHS, and diff. Common real causes seen in this codebase's own history: missing `temporary_syirkah_funds` mapping for a sharia-linked bank, or discontinued operations not being added back into the net profit reconciliation. Check whether `reconcile_variance_with_llm()` was even attempted — it only runs if an `llm_client` was passed into `validate_facts()`, which is not always wired up in every call site, so a "hard failure with no

reconciliation attempt logged" is a different bug than "reconciliation was tried and still failed".

## 11.5 LLM/API errors

| Symptom | Likely cause & fix |
|---|---|
| `401`/`403` raised immediately (not retried) | By design (`src/llm/client.py`): these are treated as fatal auth/credit errors and fail fast rather than retrying — check your API key and provider billing/credits. |
| `429`/`500`/`502`/`503`/`504` | Automatically retried with exponential backoff up to `MAX_RETRIES` (default 3). If it still fails after that, check the provider's status page or your rate limits. |
| Response truncated warning in logs | Hit the `max_tokens` ceiling (32,768 by default for statement extraction, chosen specifically to fit a 150-line-item, 9-page mega-bank balance sheet). If a genuinely larger table is truncated, consider narrowing the `--sections` range or raising `max_tokens`. |
| Structured output validation kept failing even after the one auto-repair round-trip | Read the raised `ValidationError` — it names the exact Pydantic field. Often the LLM used a concept ID that wasn't in the injected candidate list, or left a required field null. Consider trimming the schema or improving the prompt's few-shot examples for that field. |

## 11.6 "Model not found" / provider mismatch errors

`src/config.py` auto-detects the provider from whichever API key is present, in priority order: OpenAI → Gemini → OpenRouter → a hard-coded free OpenRouter default. It also silently "corrects" the model name if it doesn't look like it belongs to the chosen provider (e.g. it'll swap a Gemini-looking model name for `gpt-4o-mini` if the provider resolved to OpenAI). If extraction is quietly using a model you didn't expect, print `src.config.LLM_PROVIDER`, `DEFAULT_MODEL`, and `FALLBACK_MODEL` at the top of your debug session to see what was actually resolved, and double check your `.env` file's `LLM_PROVIDER`/`DEFAULT_MODEL`/`FALLBACK_MODEL`/`*_API_KEY` values.

## 11.7 DuckDB lock errors

```
duckdb.IOException: Could not set lock on file "data/idxfin.duckdb": Resource temporarily unavailable
```

This means something else already has the file open for writing. Checklist:

- Is a Jupyter notebook (`analysis.ipynb`) or a stray Python REPL still holding a non-read-only connection open?

- Are you running the CLI directly at the same time a Celery `duckdb_writer` worker is active? They both use the same `FileLock(data/idxfin.duckdb.lock)`, so this should queue rather than crash — but if the lock file itself got stuck (e.g. after a killed process), you may need to remove `data/idxfin.duckdb.lock` once you're sure no process actually holds it.
- For read-only exploration (Wren Engine, ad-hoc analysis, dashboards), always connect with `duckdb.connect(path, read_only=True)` — multiple concurrent readers are fine, it's concurrent *writers* that are the problem.

## 11.8 Celery/Redis issues

| Symptom | Check |
|---|---|
| `--async` extraction hangs at "QUEUED" forever | Is a worker actually running? `python3 -m src.cli worker` or check `docker-compose ps`. Also confirm Redis is reachable: the CLI itself pings Redis before enqueuing and fails fast with a clear message if it can't connect. |
| Task status stuck on `PENDING` | `PENDING` in Celery can mean either "genuinely queued" or "task ID doesn't exist / was never actually submitted" — double check the `task_id` was copied correctly. |
| Extraction succeeded but data never appears in DuckDB | Check whether `auto_ingest` was true, and separately check the `duckdb_writer` queue/worker — the extraction step and the ingestion step are two separate Celery tasks/queues; the first can succeed while the second is still waiting or has failed independently. Use `GET /api/v1/tasks` to see queue depth per queue. |

## 11.9 Testing checklist before you trust a change

```
python3 -m unittest discover tests -p "test_*.py"
```

> **Documentation drift, worth knowing:** the README states "60 automated tests" and `docs/DESIGN_LOG.md` says "49 automated tests" — but counting the actual `def test_*` functions in the `tests/` folder today gives **88**. This isn't a contradiction to panic about; it just means the test suite has grown since those documents were last updated. The lesson for you as an engineer on this project: *trust the code over the docs for exact counts*, and if you add tests, consider it good practice to bump these numbers in the README too so the next person doesn't have to discover the drift the hard way.

Two test files specifically depend on *live infrastructure* rather than pure logic, so keep that in mind when they fail:

- `tests/test_wren_semantic_contract.py` opens the real `data/idxfin.duckdb` read-only and (for the `wren_toolkit_*` tests) actually imports `wren_langchain` and queries the live Wren project

— these will fail in an environment that hasn't run any extractions yet or doesn't have Wren dependencies installed.

- `tests/test_live_llm_e2e_10_issuers.py` and `tests/test_staging_and_celery.py` likely exercise real LLM calls and/or Redis — expect these to need real API keys / a running broker, and to cost real tokens when run.

# 12. Interview Preparation Guide

Below are the questions most likely to come up if you present this project, grouped by theme, each with a plain-language model answer grounded in the actual code (not the marketing version).

## 12.1 System & architecture design

Q: Why did you choose a Medallion (bronze/silver/gold) architecture instead of one big table?

A: Each layer has a different job and a different trust level. Bronze is "what did we literally receive" (immutable, re-extractable). Silver is "the cleaned, normalized truth, one row per fact, with full lineage". Gold is "pre-joined, pre-aggregated views shaped for how people actually ask questions". Splitting them means I can fix a Gold view's SQL in seconds without re-running any LLM extraction, and I can always answer "why do you believe this number" by walking back down to Silver's `lineage_page` and then to Bronze's original page text.

Q: Why is the fact table so generic (issuer, concept, period, dims → value) instead of one column per metric?

A: Because the alternative doesn't scale across industries. A wide table needs a schema migration every time a new industry (insurance, a new bank product, a new mining KPI) shows up. This design borrows XBRL's data model: everything is one atomic fact, and industry variety lives in the *taxonomy dictionary* (data), not in database columns (schema). Adding nickel-mining or geothermal-energy support meant adding rows to `taxonomy_seed.yaml`, not writing any DDL.

Q: Walk me through why DuckDB needed a two-phase, Parquet-staged write path instead of writing directly.

A: DuckDB is an embedded, single-writer database — like SQLite, it's a library, not a server, so two processes can't hold a write lock at once. But I still want 6 workers running LLM extractions in parallel, since that part is I/O-bound and embarrassingly parallel. So I split "expensive, parallel work" (extract & normalize to Parquet, no DB access) from "cheap, serialized work" (load Parquet into DuckDB, one worker only, protected by a file lock). Parquet loading with DuckDB's `read_parquet()` is sub-50-millisecond, so serializing that part costs almost nothing, while the expensive LLM calls stay fully parallel.

Q: What would break first if you tried to scale this to 1,000 filings a day?

A: Most likely the single `duckdb_writer` worker becomes a throughput ceiling once enough extraction workers are feeding it (though at <50ms per Parquet load it would take a lot of volume to actually saturate it), and secondly the concept mapper's LLM fallback path for genuinely novel labels — that's a serial, per-label API call. In practice, the "read-only replica" pattern mentioned in the Wren

AI implementation plan (copy/checkpoint DuckDB to a separate serving file) would also become necessary so that heavy analytical/chatbot query load never competes with the ingestion writer for I/O.

## 12.2 Data engineering

Q: How do you guarantee idempotency — what happens if you accidentally run extraction twice on the same PDF?

A: The document's own `doc_id` is a SHA-256 hash of the file bytes, and every fact's `fact_id` is a deterministic hash of `(doc_id, concept, period_end, dim_key, scenario, raw_text)`. Because the hash only depends on the content, re-running produces identical IDs, and every insert uses `INSERT OR REPLACE`, so re-running is a safe upsert, not a duplicate.

Q: How do you handle restated / re-issued financial figures?

A: Nothing is ever overwritten or deleted in Silver — every occurrence of a number across every filing that ever reported it is kept as its own row, tagged with `knowledge_ts` (when we learned it) separate from the period it describes. The Gold layer then has two competing views on top of the same rows: `fact_latest` (best-known value, for fundamental analysis) and `fact_first_reported` (earliest-known value, for point-in-time/backtesting analysis, so you don't accidentally use future knowledge in a historical backtest).

Q: Why keep both cumulative (YTD) and derived discrete-quarter figures instead of just discrete quarters?

A: Indonesian interim filings are legally reported as cumulative year-to-date numbers (Q1=3 months, H1=6 months, 9M=9 months) — that's literally what's printed and audited. Discrete quarters are a *derived*, secondary view (subtracting consecutive cumulative periods) computed in the Gold layer, and the subtraction logic deliberately refuses to produce a quarter if its predecessor is missing, and flags the derived Q4 specially because it silently absorbs year-end audit adjustments.

## 12.3 ML / LLM engineering

Q: What's your core strategy for preventing LLM hallucination in a financial data pipeline, where being wrong is expensive?

A: Three layers, stacked. First, structural: force the model into a strict Pydantic schema so it physically cannot return a malformed shape. Second, vocabulary: inject the actual list of valid taxonomy concept IDs into the prompt so the model classifies into a closed set instead of inventing IDs. Third, and most important: *every single numeric value is independently verified* against the source PDF's raw page

text (the "grounding" rule) — if the exact string isn't really printed there, the fact is flagged as failed, regardless of how confident the model sounded.

Q: Why not just let the LLM compute the totals and ratios itself?

A: Because LLMs are unreliable at exact arithmetic at scale, and because "the balance sheet balances" is a hard business requirement, not a nice-to-have. The design principle is "the LLM finds and labels, Python computes and verifies" — number parsing, scale application, and every accounting equation are pure deterministic Python functions with their own unit tests, completely independent of any LLM call.

Q: You have an "LLM reconciles balance sheet variances" feature — doesn't that reintroduce the exact hallucination risk you just described?

A: It would, if the LLM's word were taken at face value — but it isn't. When a hard rule fails, the LLM is only asked to *identify* a specific named reconciling line item and value from the actual cited pages (e.g. "Temporary Syirkah Funds, IDR 10,632,695 million"). Python then independently re-computes whether applying that exact value actually closes the gap to within tolerance. If it doesn't, the suggestion is rejected and the rule stays failed. The LLM proposes; deterministic arithmetic disposes.

Q: How does the concept mapper avoid needing a large labeled training set?

A: It's a hybrid, three-tier system, not a trained model. Tier 1 is a hand-curated dictionary of ~250 known label → concept mappings plus the 148 taxonomy labels (instant, free, O(1)). Tier 2 is fuzzy token-overlap matching for near-miss phrasing. Tier 3, only for genuinely new labels, is an LLM given ~15-25 few-shot examples and asked to classify by analogy — and its answer gets *persisted* back into the dictionary, so the system effectively "learns" a new mapping once and then never needs to ask an LLM about that exact label again.

## 12.4 Domain / accounting knowledge

Q: Why does the balance sheet identity sometimes need a third term, Temporary Syirkah Funds?

A: Under Indonesian Sharia accounting (PSAK 101), Islamic banking products (e.g. BCA's sharia subsidiary inside BBCA) can't be classified as a pure liability or pure equity under Islamic finance principles, because depositors share in profit/loss rather than being owed a fixed interest-bearing debt. So it's presented as its own category, between liabilities and equity: `Assets = Liabilities + Temporary Syirkah Funds + Equity`. If you don't model this term, a sharia-linked bank's balance sheet will never mathematically balance, even though the filing itself is completely correct.

Q: Why can't you just SUM revenue across all 14 companies in a query?

A: Because several of them (ADRO, ARCI, BREN, ABMM, ESSA) report in USD while the rest report in IDR, and directly summing different currencies produces a meaningless number. Any cross-company aggregation must either filter to one currency or explicitly convert through the `fx_rate` model first — and even then, you need to pick the right rate: period-end spot rate for balance-sheet (instant) items, yearly-average rate for income-statement (duration) items.

## 12.5 Wren AI / GenBI specific

Q: What is a "semantic layer" and why is it better than raw text-to-SQL?

A: A semantic layer (here, Wren's MDL) is a curated, documented abstraction between the AI and the raw warehouse: instead of showing the model 40+ raw tables and hoping it writes correct joins and business formulas from scratch every time, you show it a handful of well-described models with pre-defined relationships and pre-defined calculated metrics (ROE, LDR, etc.). This turns "generate correct SQL and correct business logic simultaneously, from scratch, every single query" into the much easier problem of "pick the right pre-vetted model, column, and metric".

Q: What is a ReAct agent, in your own words?

A: "ReAct" = Reason + Act. Instead of answering in one shot, the model is allowed to alternate between thinking and calling tools: it can decide "I need data first", call a query tool, read the real result back, and only then compose its final answer — possibly repeating that cycle a few times. `langgraph.prebuilt.create_react_agent` implements exactly this loop for me; I just supply the model, the available tools, and a system prompt.

Q: Why maintain both a lightweight Python chat agent AND a full Docker microservice stack for the same idea?

A: Different audiences, different needs. `chat_agent.py` is zero-infrastructure and great for engineers iterating quickly or for a CLI-first workflow. The Docker stack (Wren UI + Engine + AI Service + Qdrant + Ibis) is a real multi-user web product with a visual modeler, dashboards, and a vector-indexed few-shot memory — appropriate once you want non-engineers using it day to day.

## 12.6 Testing & reliability

Q: How do you test a pipeline whose core step calls a paid, non-deterministic LLM API?

A: Split the tests by what they actually need. Pure logic (number parsing, scale math, validation-rule arithmetic, Jaccard label matching) is unit-tested with zero external dependency and zero cost. Integration correctness (does the balance sheet reconcile, are 14+ issuers all present, do Gold views have the right shape) is tested against the already-populated DuckDB warehouse, which is fast and

free once the data exists. Only a small, explicitly-named subset (`test_live_llm_e2e_10_issuers.py`) actually re-runs real LLM calls end-to-end, and that's treated as an expensive, occasional check, not part of the fast everyday test loop.

Q: What's a "golden query" and why keep them as YAML instead of just Python assertions?

A: A golden query is a hand-verified (question, correct SQL) pair. Keeping them in `golden_queries.yaml` lets the exact same file serve two purposes at once: they're fed to the AI as few-shot examples (product value) and they're executed as regression assertions in the test suite (engineering value) — one source of truth instead of two copies that could drift apart.

# 13. Glossary of Terms

| Term | Plain-English meaning |
|---|---|
| IDX / BEI | Indonesia Stock Exchange (Bursa Efek Indonesia) — where the companies in this project are listed. |
| Emiten / Issuer | A publicly listed company required to file these reports. |
| LK (Laporan Keuangan) | Financial Statement filing — the audited numbers (balance sheet, income statement, cash flow). |
| AR (Laporan Tahunan) | Annual Report — the big narrative document (400+ pages) with management commentary, KPIs, governance, and the audited LK reprinted at the back. |
| Fiscal periods: FY, Q1, H1, 9M | Full Year, first Quarter (3 months), Half year (6 months), Nine Months. Note interim periods (Q1/H1/9M) are *cumulative* year-to-date, not standalone quarters. |
| PSAK | Indonesian accounting standards (roughly the local equivalent of IFRS, and largely converged with it). |
| PSAK 101 / Syirkah Temporer | The Sharia (Islamic finance) accounting standard requiring "Temporary Syirkah Funds" to sit between liabilities and equity on the balance sheet. |
| Fact | One atomic, timestamped, sourced numeric data point — the universal unit of storage in the Silver layer. |
| Concept / Concept ID | A standardized meaning for a number, like `fs.bs.total_assets`, independent of how any one company chose to label it in their own filing. |
| Grounding | The rule that an extracted value's raw text must literally appear on its cited PDF page — the main anti-hallucination check. |
| Provenance / Lineage | The trail proving where a stored value came from: which document, which page, which extractor, which model, which run. |
| Medallion Architecture | The Bronze → Silver → Gold layering pattern for progressively cleaner data. |
| Bitemporal | Tracking two independent timelines at once: the period a fact *describes* (valid time) and when it was *learned* (knowledge time) — needed to handle restatements correctly. |

| | |
|---|---|
| Discrete vs. cumulative quarter | Cumulative = year-to-date as actually filed (9M = 9 months of activity). Discrete = one standalone quarter, derived by subtracting the prior cumulative period. |
| DuckDB | An embedded, single-process, columnar analytical database — like "SQLite for analytics". Fast for reads, but only one writer at a time. |
| Parquet | An efficient, column-oriented file format for storing tabular data, used here as a safe intermediate "staging" format before loading into DuckDB. |
| Celery | A Python distributed task queue framework, used here to run extraction jobs across multiple worker processes. |
| Redis | An in-memory data store, used here as the message broker that holds Celery's job queue. |
| Pydantic | A Python library for defining data schemas with automatic validation — used to force LLM output into a guaranteed shape. |
| MDL (Modeling Definition Language) | Wren AI's YAML-based way of describing your data warehouse's tables, columns, relationships, and calculated metrics for an AI to consume safely. |
| GenBI | Generative Business Intelligence — using generative AI to let people query analytics data in natural language. |
| ReAct agent | An AI agent pattern that alternates between "reasoning" (thinking about what to do) and "acting" (calling a tool), observing results, and repeating until it has a final answer. |
| Taxonomy | The structured dictionary of standard concept IDs, axes (dimensions), and validation rules that every extraction is mapped against (`taxonomy_seed.yaml`). |
| Hard rule vs. Soft rule | HARD a rule that must pass (zero-tolerance, e.g. balance sheet identity). SOFT a rule that's informative but doesn't block/fail the record if it doesn't hold exactly. |

# 14. Appendix: Full File Map

A single-page reference of the project layout, for quick lookup.

```
statements-extraction/
|-- schema_design/ # The "contract" layer
| |-- DESIGN.md # Full architecture rationale (time semantics, bitemporal model)
| |-- extraction_contracts.py # Pydantic v2 models every extractor must emit
| |-- storage_ddl.sql # Portable SQL DDL (ref/bronze/silver/gold)
| `-- taxonomy_seed.yaml # 148 concepts, 52 axes, 28 KPI packs, 33 validation rules
|-- src/ # Pipeline implementation
| |-- config.py # Env vars, LLM provider auto-detection, paths
| |-- pipeline.py # ExtractionPipeline orchestrator (read this first)
| |-- cli.py # inspect / extract / gold / market / api / worker / status
| |-- llm/ client.py, prompts.py
| |-- parser/ pdf_loader.py, sectionizer.py, table_parser.py
| |-- extraction/ router.py, statements.py, kpis.py, governance.py, assertions.py,
audit.py
| |-- normalization/ number_parser.py, period_resolver.py, normalizer.py
| |-- mapping/ concept_mapper.py, counterparty.py
| |-- market_data/ yfinance_loader.py
| |-- validation/ engine.py, repair.py
| |-- storage/ database.py, gold_views.py, staging.py
| |-- api/ app.py, routes.py, schemas.py
| `-- celery_app.py, tasks.py
|-- wren/ # Conversational GenBI layer
| |-- chat_agent.py # Standalone LangGraph ReAct chatbot (Section 10)
| |-- config.yaml # LiteLLM config for the Docker AI service
| |-- docker-compose.wren.yml # UI + Engine + AI Service + Qdrant + Ibis
| `-- projects/idxfin/
| |-- wren_project.yml # DuckDB data source (read-only)
| |-- relationships.yml # Model-to-model joins
| |-- models/*.yml # One semantic model per Gold view
| `-- knowledge/ # accounting_rules.yaml, golden_queries.yaml
|-- scripts/ # batch_dispatch_tickers.py, sync_market_data.py, start_workers.sh
|-- tests/ # 88 tests across 9 files (Section 11.9)
|-- data/ # idxfin.duckdb + staging/{pending,completed,failed}
|-- examples/, complete-ticker/ # Sample input PDFs for real IDX tickers
|-- docs/ # DESIGN_LOG.md and implementation plan docs
|-- docker-compose.yml, Dockerfile # Core pipeline containers
`-- analysis.ipynb # Ad-hoc DuckDB exploration notebook
```

End of document. This guide was written directly from the project's source code (not only its design docs) so that it stays honest about what actually runs today versus what is still a documented plan.